1. Уровни абстракции в вычислительной системе

Код на языке высокого уровня

↓ **Компилятор**

Программа на языке ассемблера

↓ **Ассемблер**

Машинный код

↓ **Выполнение инструкций процессором**

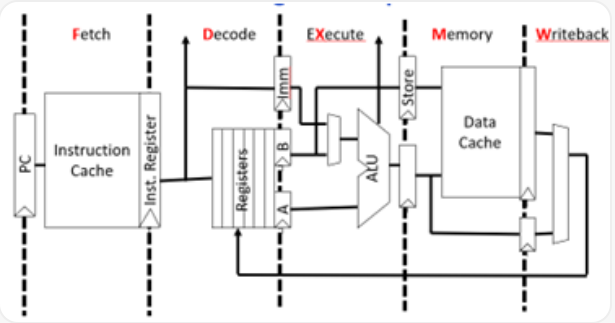
Описание аппаратной архитектуры

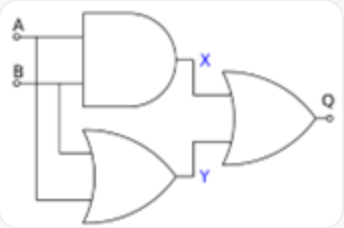
↓ **Микроархитектура**

Описание логической схемы

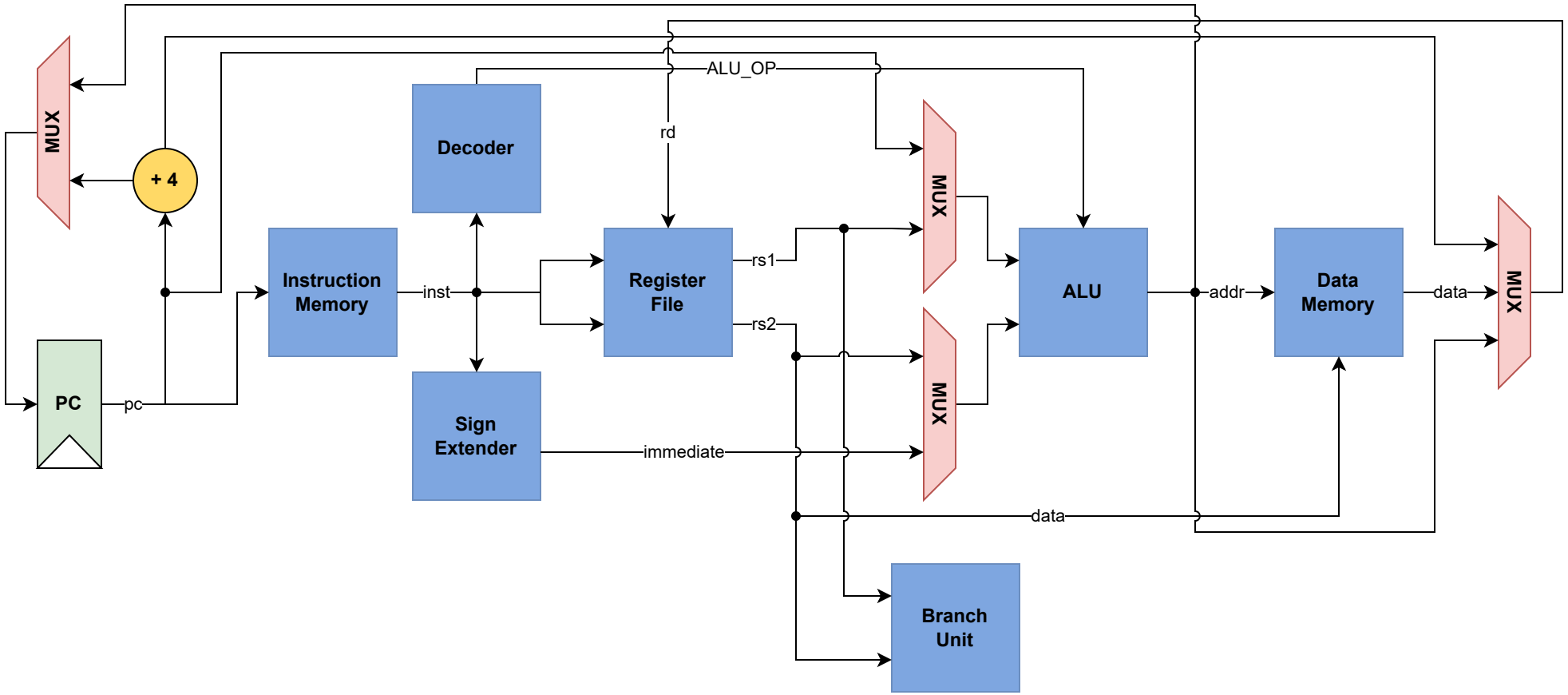
```
int add(int a, int b) {
    return a + b;
}
```

```
0x00B50533  # add a0, a0, a1
0x00008067  # ret (jalr x0, 0(ra))
```

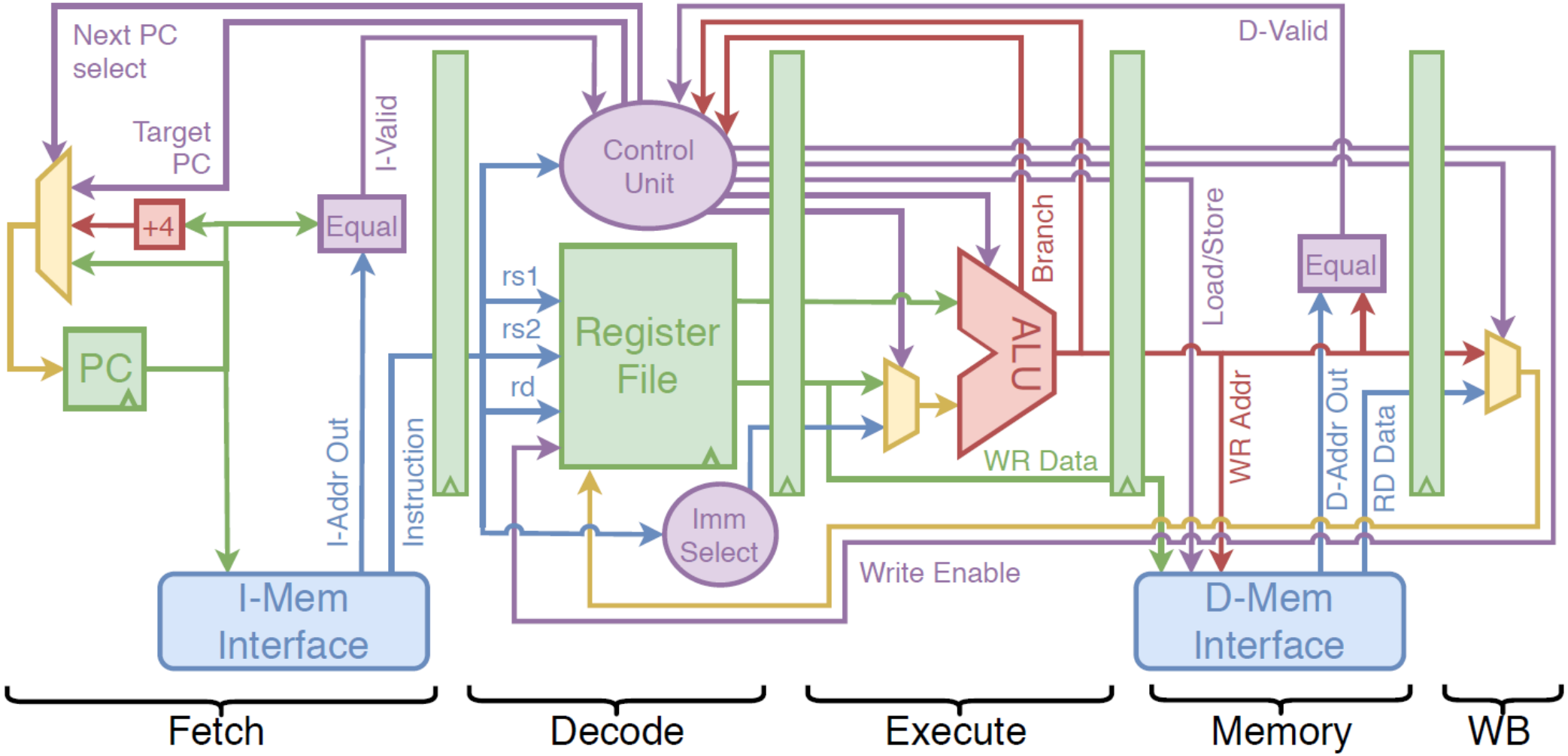




2. Модель процессора RISC-V



3. Модель процессора RISC-V с флагами



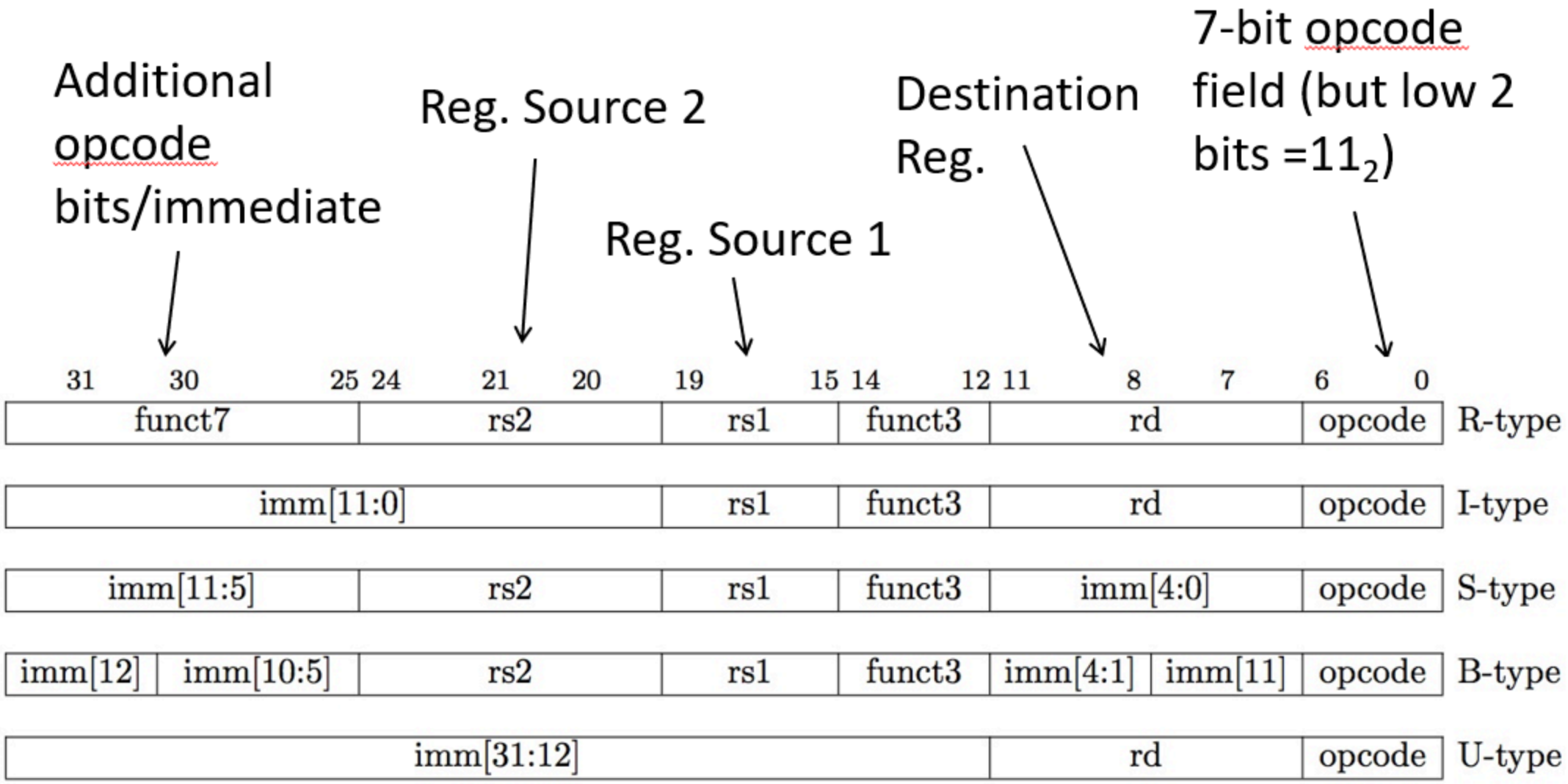
4. Базовый набор инструкций RISC-V

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
add	ADD	R	011011	0x0	0x00	rd = rs1 + rs2
sub	SUB	R	011011	0x0	0x20	rd = rs1 + rs2
xor	XOR	R	011011	0x4	0x00	rd = rs1 ^ rs2
or	OR	R	011011	0x6	0x00	rd = rs1 | rs2
and	AND	R	011011	0x7	0x00	rd = rs1 & rs2
sll	Shift Left Logical	R	011011	0x1	0x00	rd = rs1 << rs2
srl	Shift Right Logical	R	011011	0x5	0x00	rd = rs1 >> rs2
sra	Shift Right Arith	R	011011	0x5	0x20	rd = rs1 >> rs2
slt	Set Less Than	R	011011	0x2	0x00	rd = (rs1 < rs2) ? 1:0
sltu	Set Less Than (U)	R	011011	0x3	0x00	rd = (rs1 < rs2) ? 1:0
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm
slli	Shift Left Logical Imm	I	0010011	0x1	Imm[5:11]=0x00	rd = rs1 << imm[0:4]
srli	Shift Right Logical Imm	I	0010011	0x5	Imm[5:11]=0x00	rd = rs1 >> imm[0:4]
srai	Shift Right Arith Imm	I	0010011	0x5	Imm[5:11]=0x20	rd = rs1 >> imm[0:4]
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm) ? 1:0
sltiu	Set Less Than (U) Imm	I	0010011	0x3		rd = (rs1 < imm) ? 1:0
lb	Load Byte	I	0000011	0x0		rd = M[rs1 + imm][0:7]
lh	Load Half	I	0000011	0x1		rd = M[rs1 + imm][0:15]
lw	Load Word	I	0000011	0x2		rd = M[rs1 + imm][0:31]
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1 + imm][0:7]
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1 + imm][0:15]

4. Базовый набор инструкций RISC-V(2)

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm
bge	Branch >=	B	1100011	0x5		if(rs1 >= rs2) PC += imm
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm
bgeu	Branch >= (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm
lui	Load Upper Imm	U	0110111			rd = imm << 12
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)
ecall	Environment Call	I	1110011	0x0	Imm=0x0	Transfer control to OS
ebreak	Environment Break	I	1110011	0x0	Imm=0x1	Transfer control to debugger

5. Кодирование инструкций RISC-V



6. Формат инструкции R

`[31:25] funct7 | [24:20] rs2 | [19:15] rs1 | [14:12] funct3 | [11:7] rd | [6:0] opcode`

Поле	Биты	Назначение
opcode	[6:0]	класс OP (`0110011`)
rd	[11:7]	куда записать результат
rs1	[19:15]	1-й операнд (регистр)
rs2	[24:20]	2-й операнд (регистр)
funct3	[14:12]	подтип операции
funct7	[31:25]	уточнение (например add/sub, srl/sra)

7. Формат инструкции I

```
[31:20] imm[11:0] | [19:15] rs1 | [14:12] funct3 | [11:7] rd | [6:0] opcode
```

Поле	Биты	Назначение
opcode	[6:0]	класс (IMM / LOAD / JALR / SYSTEM)
rd	[11:7]	куда записать результат
rs1	[19:15]	операнд / базовый адрес
funct3	[14:12]	подтип операции
imm[11:0]	[31:20]	константа/смещение (sign-extend)

8. Формат инструкции S

`[31:25] imm[11:5] | [24:20] rs2 | [19:15] rs1 | [14:12] funct3 | [11:7] imm[4:0] | [6:0] opcode`

Поле	Биты	Назначение
opcode	[6:0]	класс STORE (`0100011`)
rs1	[19:15]	**base**: откуда берём адрес
rs2	[24:20]	**data**: что записываем в память
funct3	[14:12]	размер записи: `sb/sh/sw`
imm[11:0]	[31:25] + [11:7]	смещение (sign-extend)

9. Формат инструкции B

`[31] imm[12] | [30:25] imm[10:5] | [24:20] rs2 | [19:15] rs1 | [14:12] funct3 | [11:8] imm[4:1] | [7] imm[11] | [6:0] opcode`

Поле	Биты	Назначение
opcode	[6:0]	класс BRANCH (`1100011`)
rs1	[19:15]	1-й операнд сравнения
rs2	[24:20]	2-й операнд сравнения
funct3	[14:12]	тип условия (beq/bne/blt/...)
imm(B)	разрезан	смещение для `PC` (sign-extend)

10. Формат инструкции U

`[31:12] imm[31:12] | [11:7] rd | [6:0] opcode`

Поле	Биты	Назначение
opcode	[6:0]	класс U-type: LUI= `0110111` , AUIPC= `0010111`
rd	[11:7]	куда записать результат
imm[31:12]	[31:12]	20 бит immediate (верхняя часть)

11. Формат инструкции J

`[31] imm[20] | [30:21] imm[10:1] | [20] imm[11] | [19:12] imm[19:12] | [11:7] rd | [6:0] opcode`

Поле	Биты	Назначение
opcode	[6:0]	JAL (`1101111`)
rd	[11:7]	куда записать `PC+4` (обычно `ra`)
imm(J)	разрезан	смещение от `PC` (sign-extend)

12. Соответствие классу инструкций - блокам

Класс (RV32I)	Формат	Какие блоки используются	Что происходит
OP (`add/sub/...`)	R	RegFile → ALU → WB → RegFile	rd = f(rs1, rs2)
OP-IMM (`addi/...`)	I	RegFile → ALU(imm) → WB → RegFile	rd = f(rs1, imm)
LOAD (`lw/lb/...`)	I	RegFile → ALU(addr) → DataMem(read) → WB → RegFile	rd = Mem[rs1+imm]
STORE (`sw/sb/...`)	S	RegFile → ALU(addr) → DataMem(write)	Mem[rs1+imm] = rs2
BRANCH (`beq/...`)	B	RegFile → ALU(compare) → BranchUnit → PC MUX	PC = PC+imm или PC+4
JAL (`jal`)	J	PC+4 → WB → RegFile, PC+imm → PC MUX	rd=PC+4; PC=PC+imm
JALR (`jalr`)	I	RegFile(rs1)+imm → PC MUX, PC+4 → WB	rd=PC+4; PC=(rs1+imm)&~1
U-type (`lui/auipc`)	U	ImmGen/PC → WB → RegFile	rd = imm<<12 или PC+(imm<<12)

13. Сигналы управления

Сигнал	Куда идёт	Что делает
RegWrite	RegFile	Разрешает запись в `rd`
ALUSrc	MUX перед входом B ALU	`0`: берём `rs2`, `1`: берём `imm`
ALUOp	ALU control	Выбирает операцию ALU (add/sub/and/...) по `funct3/funct7`
MemRead	Data Memory	Чтение из памяти (load)
MemWrite	Data Memory	Запись в память (store)
MemToReg	MUX writeback	`0`: в `rd` идёт ALU result, `1`: данные из памяти
Branch	Branch Unit / PC MUX	Разрешает ветвление (используем результат сравнения)
Jump	PC MUX	Безусловный переход (`jal/jalr`)
PCSel *(или PCSrc)*	PC MUX	Выбор следующего PC: `PC+4` / target
ImmSel	ImmGen/SignExt	Выбирает тип immediate: I/S/B/U/J

14. opcode / funct3 / funct7 — три уровня кодирования

Уровень	Сколько бит	За что отвечает	Пример
opcode	7	**Класс** инструкции (формат/тип): load/store/branch/op/jump	`0110011` → OP (R-type)
funct3	3	**Подтип** внутри класса	BRANCH: `000` =beq, `001` =bne
funct7	7	**Уточнение**, когда funct3 недостаточно	OP: `add` vs `sub`, `srl` vs `sra`

Декодер сначала смотрит `opcode` (что это за класс), затем `funct3`, и при необходимости `funct7`.

15. Immediate в инструкциях I/S/B/U/J

Формат	Где используется	Размер imm (эфф.)	Что означает
I-type	`addi`, `lw`, `jalr`	12 бит	константа / смещение / base+offset
S-type	`sw`, `sb`, `sh`	12 бит	смещение для store (base+offset)
B-type	`beq`, `blt`, …	13 бит	PC-relative смещение для ветвления (**кратно 2**)
U-type	`lui`, `auipc`	20 бит	верхние 20 бит: `imm << 12`
J-type	`jal`	21 бит	PC-relative смещение прыжка (**кратно 2**)

imm иногда “разрезан” для того чтобы сохранить удобные позиции `rs1/rs2/funct3` в 32-битной инструкции